

Architecture technique - UpcycleConnect

Objectif du document

Ce document présente l'architecture technique retenue pour la réalisation de la solution UpcycleConnect. Il couvre les objectifs de la mission 1 (application web métiers), de la mission 2 (infrastructure système et réseau), ainsi que les modules retenus de la mission 3 (bloc Développement : module 1, bloc Infra : modules 1 et 2).

1. Contexte général et périmètre couvert

1.1 Missions couvertes

- **Mission 1 - Gestion des solutions applicatives**
 - Espace Particuliers (dépôt d'annonces, dépôt en conteneurs, planning, services, commandes, tutoriel, etc.)
 - Espace Professionnels/Artisans (contrats, abonnement, publicité, récupération d'objets, projets Upcycling)
 - Espace Salariés (formateurs / animateurs : planning, gestion de formations, modération, contenus)
 - Back office (administration générale, gestion des utilisateurs, validations, suivi financier)
 - **Mission 2 - Infrastructure système et réseau**
 - Architecture réseau (VLANs, DMZ, VPN, firewalls OPNSense obligatoires)
 - Continuité de service (HA, sauvegarde, supervision, ticketing)
 - Infrastructure serveur (containers, hébergement, serveurs web, bases de données)
 - **Mission 3 - Modules sélectionnés**
 - **Bloc Développement - Module 1** : Application mobile Android pour professionnels/artisans (fonctionnalités essentielles implémentées côté API et backoffice pour permettre un usage mobile via API REST).
 - **Bloc Infra - Module 1** : Sécurisation WiFi (VLAN, RADIUS/AD, normes WiFi, récupération de mot de passe) et intégration AD/RADIUS.
 - **Bloc Infra - Module 2** : Mécanismes de redondance (backups, serveurs DNS/DHCP, PRA, RODC, sauvegardes journalières et mensuelles).
-

2. Vue d'ensemble de l'architecture applicative

2.1 Architecture logique (3 couches)

L'application suit une architecture 3 couches :

1. **Frontend Web** (interface utilisateur) :

- PHP natif + HTML/CSS/JavaScript (sans framework JS/HTML).
- Deux applications distinctes :
 - **PA - Site Principal** : espace adhérents / particuliers / professionnels / salariés.
 - **PA - BO** : interface d'administration / back office.

2. Backend API :

- API REST écrite en Go (dossier **PA - API**).
- Point d'entrée unique exposé sur un port (par défaut 9999).
- Gère l'authentification, la logique métier, les accès BDD et les intégrations externes.

3. Base de données :

- MySQL relationnel (Schéma, relations, intégrité, transactions).
- Script d'initialisation : **db_schema.sql**.

2.2 Infrastructure de déploiement / containerisation

- **Docker Compose** fournit un environnement de développement reproductible (**docker-compose.yml**).
 - Services : **db** (MySQL), **api** (Go).
 - Réseau isolé (**DataNetwork**), volumes persistants.
- **Dockerfile API**
 - Multi-stage build (compilation Go → image légère **distroless/static**).
 - Permet un déploiement léger, sécurisé et déterministe.
- **Serveur web** (production) :
 - Apache + PHP (XAMPP en local, mais le déploiement doit être configuré sur un serveur Linux / cloud).
 - Réécriture d'URL pour simplifier les URLs et permettre des codes d'erreur propres.

3. Justification des choix technologiques

3.1 Pourquoi Go pour l'API ?

- **Performance** : exécutables compilés, faible latence, gestion facile des connexions simultanées.
- **Simplicité** : base de code claire, peu de dépendances externes, bonne lisibilité.
- **Déploiement** : binaire unique, packagé dans un conteneur Docker minimal (distroless).
- **Écosystème** : packages pour MySQL, JWT, OAuth, etc. (utilisation de github.com/golang-jwt/jwt/v4 pour les tokens).

3.2 Pourquoi MySQL ?

- **Intégrité** : contraintes, transactions, clés étrangères.
- **Simplicité** : SQL standard, facile à administrer.
- **Évolutivité** : indexes, réplication si besoin.
- **Structure** : relationnelle (clé <-> Valeur)

3.3 Pourquoi PHP + JS/CSS ?

- **Indépendance de frameworks** : Pas de blocages Symfony / Laravel / React / Vue.
- **Maîtrise complète** : la logique côté présentation reste simple, bien visible et modifiable sans abstractions lourdes (type M.V.C).
- **Performance** : meilleures performances, moins de complexité.
- **Compatibilité** : fonctionne sur n'importe quel hébergement PHP standard.

3.4 Pourquoi Docker ?

- **Reproductibilité** : même configuration, peu importe l'environnement.
- **Isolation** : la base de données tourne dans un container isolé.
- **Déploiement** : facilite la mise en production sur des serveurs Linux, Cloud providers supports Docker.

3.5 Pourquoi Git, Composer, NPM ?

- **Git** : gestion de versions collaborative.
- **Composer** : gestion des dépendances PHP (PHPMailer, Google API Client, etc.).
- **NPM** : gestion des dépendances JS utiles. Même si l'UI est native, NPM permet d'ajouter des outils métiers, comme pour la double authentification par code TOTP .

4. Architecture fonctionnelle (Mission 1)

4.1 Espaces utilisateur

4.1.1 Particuliers

Fonctionnalités principales implémentées ou prévues :

- Inscription, connexion, profil.
- Dépôt d'annonces (don / vente) avec validation administrative.
- Demande de dépôt d'objet dans un conteneur.
 - Génération d'un code d'ouverture (workflow prévu côté gestion).
 - Génération d'un code-barres pour récupération par professionnels.
- Consultation du planning (cours, ateliers, services).
- Catalogue services/formations/événements (avec paiement via Stripe).
- Score Upcycling (calcul basé sur poids et type de matériau).
- Tutoriel de première connexion (UI en overlay bloquant). Cette fonctionnalité est prévue : la réalisation reste à implémenter complètement.

4.1.2 Professionnels / artisans

- Gestion des contrats / abonnements / facturation (Stripe Subscriptions, Webhooks).
- Accès aux annonces (avec achat possible).
- Gestion de la récupération des objets en conteneurs.
- Gestion et promotion de projets d'upcycling.

4.1.3 Salariés (animateurs et formateurs)

- Création et animation de formations (workflow avec validation d'un responsable).
- Gestion des plannings et des sessions.
- Publication de conseils.
- Modération des forums et contenus.

4.1.4 Back office d'administration

- Gestion complète des utilisateurs (rôles, statuts, bannissements).
- Validation des annonces et des dépôts.
- Suivi financier (revenus, commissions, abonnement).
- Gestion des notifications.
- ...

4.2 Exposition de l'API REST

- Endpoint de base : `/` (santé, documentation interne).
- Routes démultipliées selon les ressources : `/users`, `/orders`, `/annonces`, `/services`, `/subscriptions`, etc.
- Authentification : JWT portés dans l'en-tête `Authorization: Bearer <token>`.
- Rôles : contrôlés via middleware (`RoleMiddleware`) et claims JWT.
- Protection des routes sensibles (accès admin, modification de contenus).
- Mécanisme de validation des payloads côté API.

4.3 Schéma de données (extrait)

La base actuelle est centrée sur la table `users` (cf. `docs/DATABASE.md`). Elle utilise :

- Identifiants UUID (`CHAR(36)` en base, générés via `uuid.New()` en Go).
- Email et username uniques.
- Hachage de mot de passe (bcrypt, via `golang.org/x/crypto/bcrypt`).

Des tables supplémentaires sont prévues pour :

- `annonces` (déclarations de dons/ventes).
- `orders` (commandes, paiements).
- `services` (catalogue, formations, événements).
- `containeurs` (gestion de dépôt / codes).
- `planning` (sessions, réservations).
- et bien d'autres ...

Ces tables sont créées via `db_schema.sql`.

5. Sécurité et gestion des accès

5.1 Authentification

- **JWT** : génération de token signé (`HS256`) avec un secret stocké en variable d'environnement (`JWT_SECRET`).
- **OAuth** : intégration Google et Microsoft (client ID / secret stockés en `.env`).

- **Sessions PHP** : côté frontend, gestion de sessions avec cookies sécurisés.

5.2 Entités sensibles

- **Mot de passe** : hachage bcrypt (Go) + validation côté API.
- **Clés secrètes** : jamais committées (exemple `.env` non suivi). Le README indique les variables attendues.
- **Webhooks Stripe** : validation par signature (secret de webhook).

5.3 Protection des formulaires

- reCAPTCHA v3 est intégré (clé côté client + secret côté serveur) pour limiter les bots.
- Validation côté serveur sur tous les payloads JSON/formulaires (taille, types, contraintes).

5.4 Sécurité réseau (Mission 2)

- Architecture à trois zones : **LAN, DMZ, Internet**.
 - **Firewalls OPNSense (HA)** : obligatoires. Ils filtrent le trafic inter-VLAN et vers la DMZ.
 - **VPN** : accès distant sécurisé (OpenVPN ou WireGuard) pour télétravail.
 - **VLANs** : séparation des postes (Direction, Marketing, Commercial, RH, Informatique, HManager, Régional, DMZ).
 - **DMZ** : serveurs exposés : API (Go), site web, serveur mail, reverse proxy, etc.
-

6. Intégrations externes

6.1 Paiements - Stripe

- Clés Stripe (`STRIPE_PUBLISHABLE_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`) configurées dans les `.env`.
- Le backoffice et le frontend utilisent Stripe pour :
 - Gestion des abonnements (Stripe Subscriptions)
 - Paiement de services / formations
 - Facturation et commissions

6.2 Visio - Zoom

- Variables d'environnement prévues pour Zoom (`ZOOM_CLIENT_ID`, `ZOOM_CLIENT_SECRET`, `ZOOM_ACCOUNT_ID`, `ZOOM_USER_ID`).
- Permet de générer des liens de visioconférence pour les formations / ateliers.

6.3 Géolocalisation d'adresses (API gouvernementale)

- Adresse : <https://api-adresse.data.gouv.fr/search/>.
- Utilisée par l'interface pour l'autocomplétion des adresses (déclaration de dépôt, planning, etc.).

6.4 OAuth (Google / Microsoft)

- Utilisé pour l'authentification des utilisateurs via des comptes externes.
- Permet une entrée simplifiée pour les adhérents et réduit le besoin de gestion de mot de passe.

6.5 Notifications (planifiées)

- La documentation du sujet mentionne OneSignal / push notifications.
- Le code existant n'inclut pas encore cette intégration, mais l'architecture API prend en charge l'envoi de notifications via un service tiers (p. ex. OneSignal, Firebase Cloud Messaging) en ajoutant un module dans **PA - API/app/**.

6.6 Gemini (IA)

- Module de modération intelligente basé sur un modèle de type LLM.
 - Détection de contenu potentiellement généré par une IA (texte / messages / publications).
 - Estimation de l'impact environnemental (Upcycle Score).
-

7. Déploiement et exploitation

7.1 Environnement local

- Installer XAMPP ou équivalent (Apache + PHP + MySQL).
- Démarrer les services Apache et MySQL.
- Importer la base : `mysql -u root -p < db_schema.sql`.
- Configurer les `.env` côté PHP (**PA - Site Principal/.env** et **PA - BO/.env**) et côté API (**PA - API/.env**).
- Lancer l'API Go :

```
cd "PA - API"
go run .
```

- Accéder au site PHP via Apache (ex.: `http://localhost/PA/PA%20-%20Site%20Principal/pages/public/index.php`).

7.2 Déploiement en production

- Héberger dans un environnement Linux (Ubuntu/Debian ou cloud provider).
- Déployer avec Docker (recommandé) :
 - Lancer les services via `docker-compose up -d`.
- Configurer un reverse-proxy (NGINX / Apache) pour exposer les sites web et l'API.
- Mettre en place un certificat TLS (Let's Encrypt).
- Configurer la réécriture d'URL pour obtenir des URLs propres (Apache `mod_rewrite` ou NGINX `try_files`).

7.3 Continuité de service et supervision (Mission 2 / Module Infra 2)

- **Sauvegardes** : backup complet et incrémental (Veeam, Acronis, ou solution open source). Exports réguliers de la base MySQL (`mysqldump`).

- **Supervision** : installer Zabbix / Nagios. Le système doit monitorer l'API, la base, l'état des serveurs, les firewalls et l'adresse IP publique.
 - **Ticketing** : GLPI (gestion de parc et tickets). Intégration possible via l'API GLPI pour centraliser les incidents.
 - **Redondance** : prévoir un second serveur de base de données (réplication), un second container API, et un second serveur web.
 - **RODC** : serveur de domaine en lecture seule pour DRP (plan de reprise d'activité). (Note : la partie Active Directory réelle n'est pas implémentée dans le code PHP/Go, c'est une préconisation d'architecture pour permettre la mise en place de GPO mentionnée dans le sujet.)
-

8. Annexe : Emplacements clés dans le code

- **API Go** : PA - API/api.go, PA - API/app/, PA - API/db/.
- **Frontend PHP** : PA - Site Principal/pages/, PA - BO/pages/.
- **Configuration** : .env sous chaque application (PA - API/.env, PA - Site Principal/.env, PA - BO/.env).
- **Schéma BDD** : db_schema.sql, docs/DATABASE.md.
- **Docker** : docker-compose.yml, PA - API/Dockerfile, PA - Site Principal/Dockerfile, PA - BO/Dockerfile.

Voir le fichier *FileStructure.pdf* ou *FileStructure.md* pour plus de clarté La documentation complète du projet est disponible sur le dépôt [GitHub](#)

Ce document est à jour de l'état actuel du code et des choix techniques du projet UpcycleConnect. Il doit être mis à jour en cas d'évolution majeure de l'architecture.